



LangChain: Building Intelligent LLM Applications

From RAG & Prompts to Production-Ready Systems

Why LangChain?



The Problem



Manual RAG pipelines are complex



Hard to manage memory



Difficult to add tools & agents



Not production ready



The Solution



**Orchestration Framework
for LLM Apps**

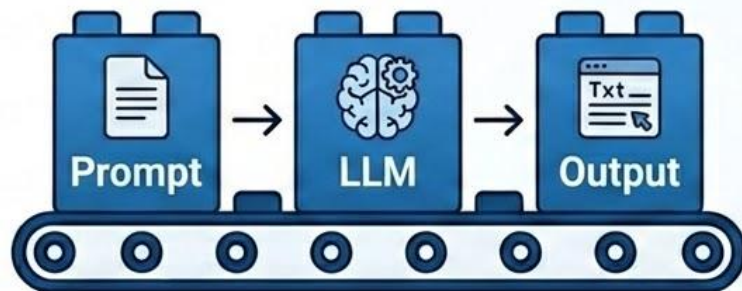
The LangChain Ecosystem



Core Philosophy of LangChain

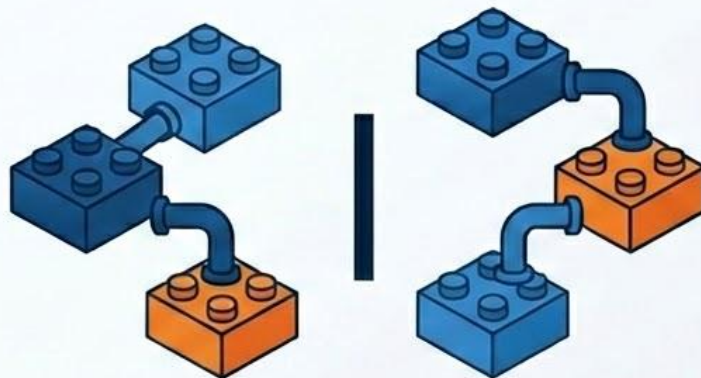


Chains



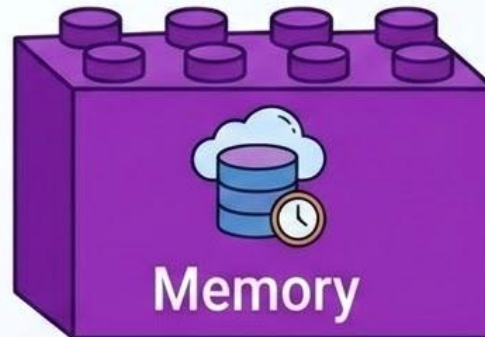
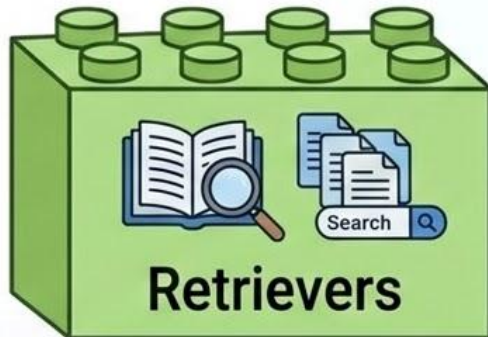
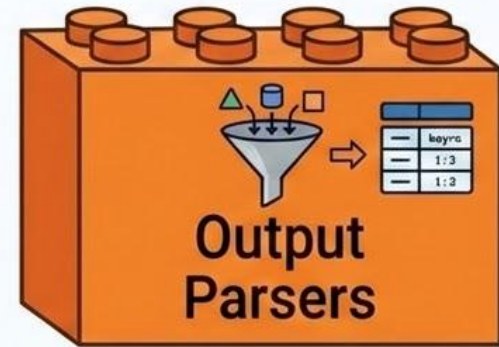
Sequence of steps
(Prompt → LLM → Output).

LCEL (LangChain Expression Language)

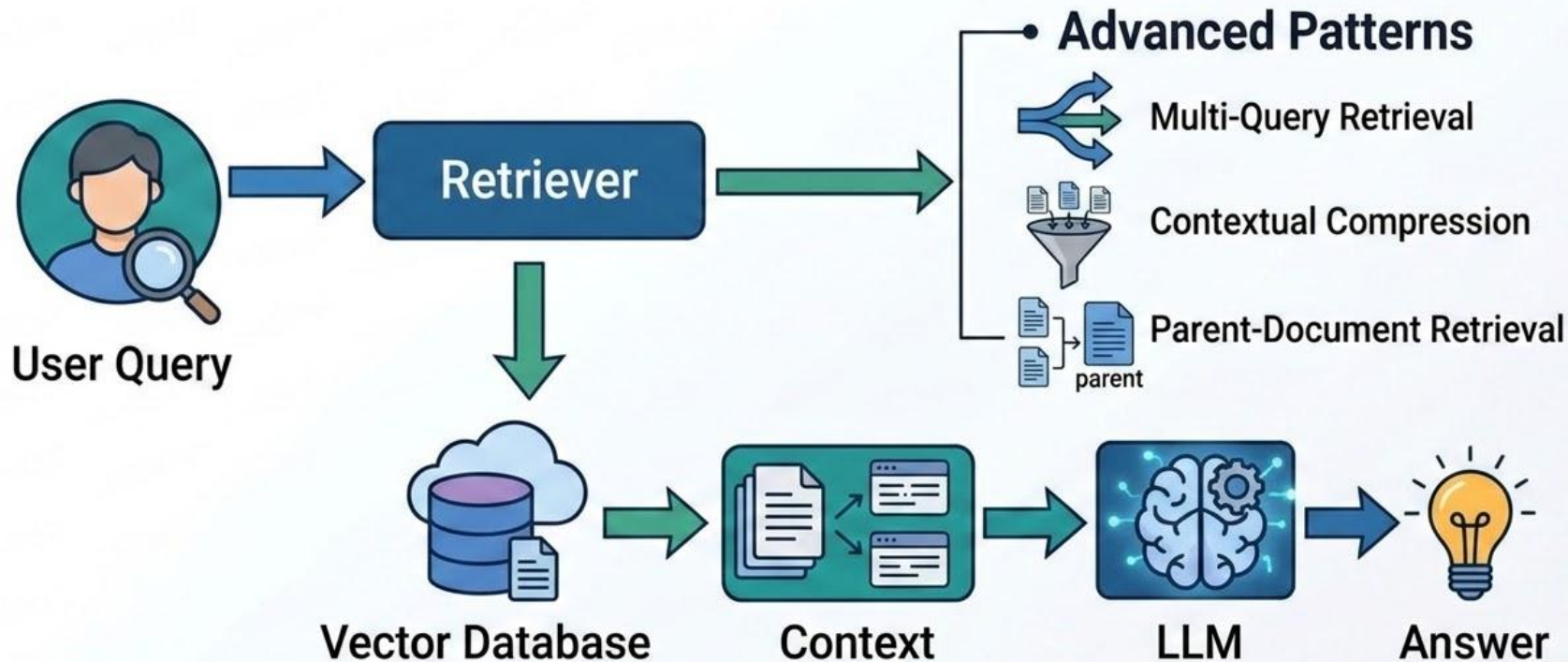


Modern pipeline approach
with pipe symbol.

LangChain Components – Lego Blocks

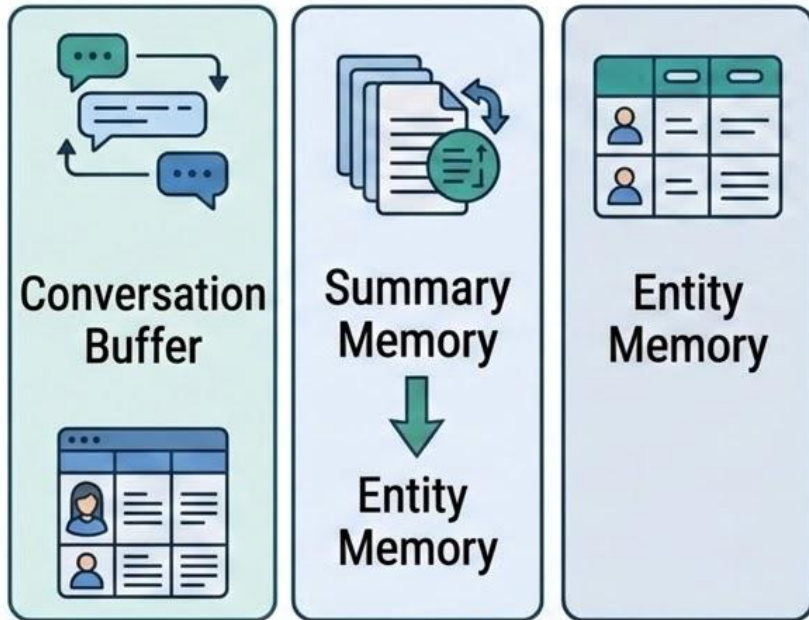


LangChain RAG Implementation in LangChain

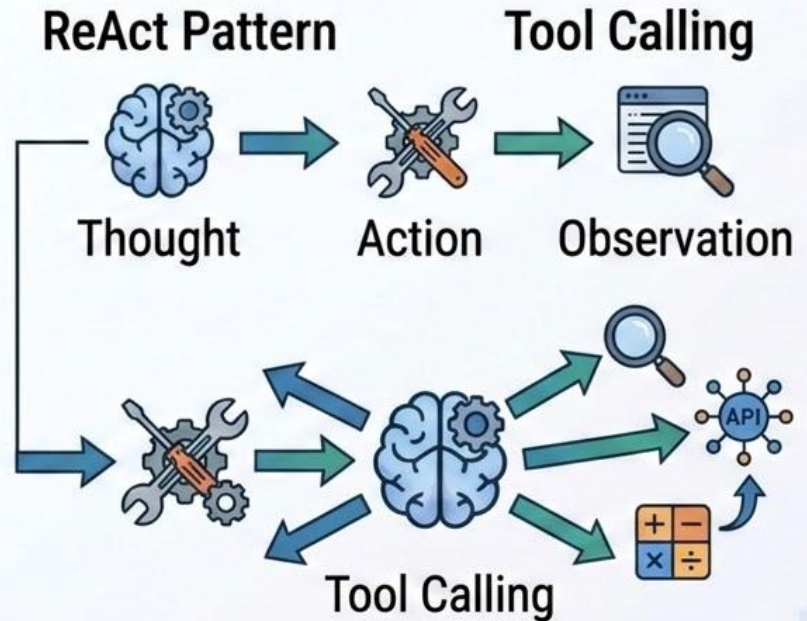


LangChain Memory + Agents

Memory Types

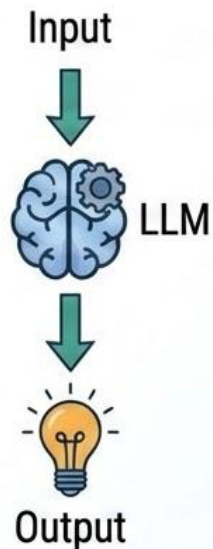


Agents

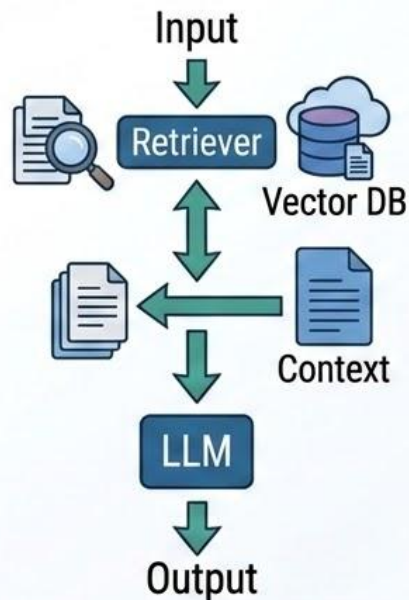


Common Application Patterns

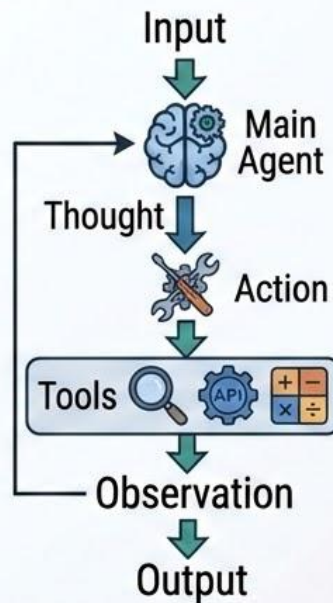
1. Simple Chain



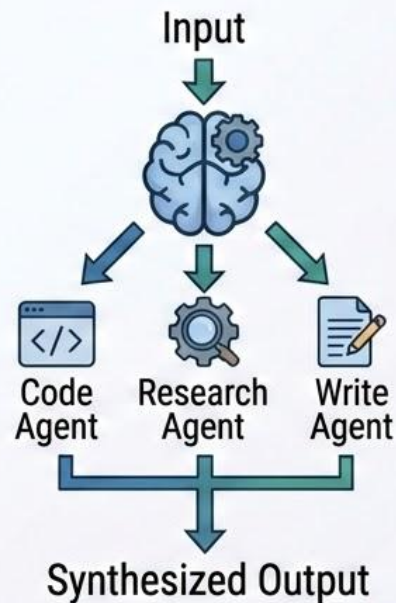
2. RAG Chain



3. agent with tools



4. Multi-Agent System



LangChain – Pros & Cons

Pros



Rapid Development
(Allows for fast iteration and prototyping)



Huge Ecosystem
(Integrations with many tools, data sources, and models)



Production Tools
(Features for tracing, evaluation, and monitoring)

Cons



Abstraction Complexity
(Hard to understand inner workings, “leaky abstractions”)



Debugging Difficulty
(Tracing execution paths through multiple steps is complex)



Performance Overhead
(Multiple abstraction layers can introduce latency)

When to Use LangChain



Complex Workflows: For multi-step tasks across diverse sources.



Advanced AI Agents: For agentic, RAG, or multi-model systems.



Persistent Context: When maintaining deep interaction history.



Abstracted Interface: To avoid vendor lock-in and switch models.

Key Takeaways

- LangChain multiplies your RAG knowledge.
- Think in components not code.
- LCEL is the future.
- Use LangSmith for debugging.
- Focus on architecture not just implementation.

Build the next generation of AI applications with LangChain.



Thank You! Questions?

